



Subtopic: Interrupts and I/O Structures

1. Interrupts

Definition

An **interrupt** is a mechanism that temporarily halts the normal execution of a CPU program to give attention to a higher-priority event, after which execution resumes from where it left off.

In other words, interrupts allow the CPU to **respond to asynchronous events** (like input from a keyboard, completion of I/O, or error conditions).

1.1 Types of Interrupts

1. Hardware Interrupts:

- Generated by external hardware devices.
- Examples: Keyboard key press, mouse click, I/O completion.

2. Software Interrupts:

- Triggered by executing special instructions in software.
- Used for system calls, debugging, or exceptions.

3. Maskable vs. Non-Maskable Interrupts (NMI):

- *Maskable Interrupts*: Can be ignored or delayed by the CPU.
- *Non-Maskable Interrupts*: Cannot be ignored (used for critical errors like hardware failures).

4. Vectored vs. Non-Vectored Interrupts:



- *Vectored*: Interrupt handler address is supplied by hardware automatically.
 - *Non-Vectored*: CPU must determine the address of the handler.
-

1.2 Interrupt Cycle

When an interrupt occurs:

1. CPU **completes** the current instruction.
 2. CPU **saves** its current state (program counter, registers).
 3. CPU **jumps** to the appropriate **Interrupt Service Routine (ISR)**.
 4. ISR handles the interrupt (e.g., process input, send acknowledgment).
 5. CPU **restores** saved state and resumes execution.
-

1.3 Advantages of Interrupts

- Efficient use of CPU (no need for constant polling).
 - Immediate response to high-priority events.
 - Supports multitasking and I/O concurrency.
-

2. Input/Output (I/O) Structures

Definition



I/O structures define how data is transferred between CPU, memory, and external devices. Since I/O devices are much slower than CPUs, efficient mechanisms are needed to prevent CPU from being idle.

2.1 I/O Techniques

1. Programmed I/O (Polling):

- CPU actively checks device status in a loop.
- Wastes CPU time if device is slow.
- Suitable for simple, low-speed devices.

2. Interrupt-Driven I/O:

- Devices signal CPU via interrupts when ready.
- CPU can perform other tasks while waiting.
- More efficient than polling.

3. Direct Memory Access (DMA):

- Data is transferred directly between memory and I/O device **without CPU intervention**.
 - CPU only sets up transfer, DMA controller handles the rest.
 - Suitable for high-speed transfers (e.g., disk, network).
-

2.2 I/O Bus and Controllers

- **I/O Bus:** Pathway connecting CPU, memory, and peripherals. Examples: PCIe, USB.



- **I/O Controller:** Hardware interface that manages data exchange between devices and CPU/memory.
 - Example: Disk controller, Network Interface Card (NIC).
-

2.3 I/O Structure Types

1. Memory-Mapped I/O:

- I/O devices are assigned specific addresses in the same space as memory.
- CPU uses normal load/store instructions for I/O.
- Advantage: Simpler and uniform.

2. Isolated I/O (Port-Mapped I/O):

- Separate address space for I/O.
 - Special instructions (IN, OUT) are used for data transfer.
 - Advantage: Memory space is not reduced.
-

2.4 Performance Considerations

- **Latency:** Delay between request and completion of I/O.
 - **Throughput:** Amount of data transferred per unit time.
 - **CPU Utilization:** Should remain high by minimizing CPU waiting for I/O.
-

✓ Summary

These notes were created by Muhammad Abdullah Ahsan, an admin from the GCUF GUIDERS channel. Join here: [GCUF GUIDER](#).



- **Interrupts** allow the CPU to respond to urgent events, avoiding constant polling, and are categorized into hardware/software, maskable/non-maskable, and vectored/non-vectored.
- **I/O Structures** define how data moves between CPU, memory, and devices. Methods include **programmed I/O (polling)**, **interrupt-driven I/O**, and **DMA** for high-speed transfers.
- Efficient I/O design ensures better **performance, resource utilization, and responsiveness**.